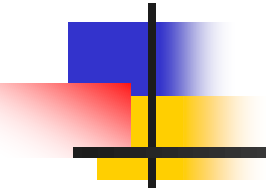
Fundamental of Software Engineering (SENG 1061)

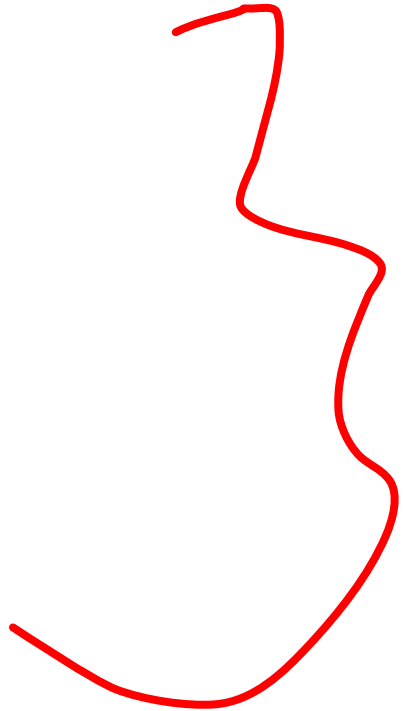


Chapter 03

Requirement Engineering

Outline

- Defining requirements of a project and documentation
- What is requirement
- Types of Requirements
  - User and System Requirements
  - Functional and non-functional requirements
- Characteristics of Requirement
- Gathering and classifying requirement
- Use case modeling
- Validation and Verification



Objectives of the Chapter

After Completing this chapter student will

- *Eliciting requirements* from the customers,
- *Reviewing requirements* to ensure their quality,
- Modelling *requirements*,
- Documenting requirements for use by the *design and test teams*.

Software Requirements

- Requirements are descriptions of the services that a software system must provide and the constraints under which it must operate
- Requirements Engineering is the process of establishing the services that the customer requires from the system and the constraints under which it is to be developed and operated
- Requirements may serve a dual function:
  - As the basis of a bid for a contract
  - As the basis for the contract itself

The Requirements Process

Why Are *Requirements Important*?

- Top factors that *caused project to fail*. (Standish Group 1994):
  - Incomplete requirements (13.1%),
  - Lack of user involvement (12.4%),
  - Lack of resources (10.6%),
  - Unrealistic expectations (9.90%), Lack of executive support (9.30%),
  - Changing requirements and specifications (8.70%),
  - Lack of planning (8.10%), System no longer needed (7.50%),
- Some part of the *requirements process* is involved in almost all of these causes.
- *Requirements error* can be expensive if *not detected early*.

Characteristics of a Good Requirement

- **Clear and Unambiguous**
  - standard structure
  - has only one possible interpretation
  - Not more than one requirement in one sentence
- **Correct**
  - A requirement contributes to a real need
- **Understandable**
  - A reader can easily understand the meaning of the requirement
- **Verifiable**
  - A requirement can be tested
- **Complete**
- **Consistent**
- **Traceable**

Conti…

Making Requirements Testable:

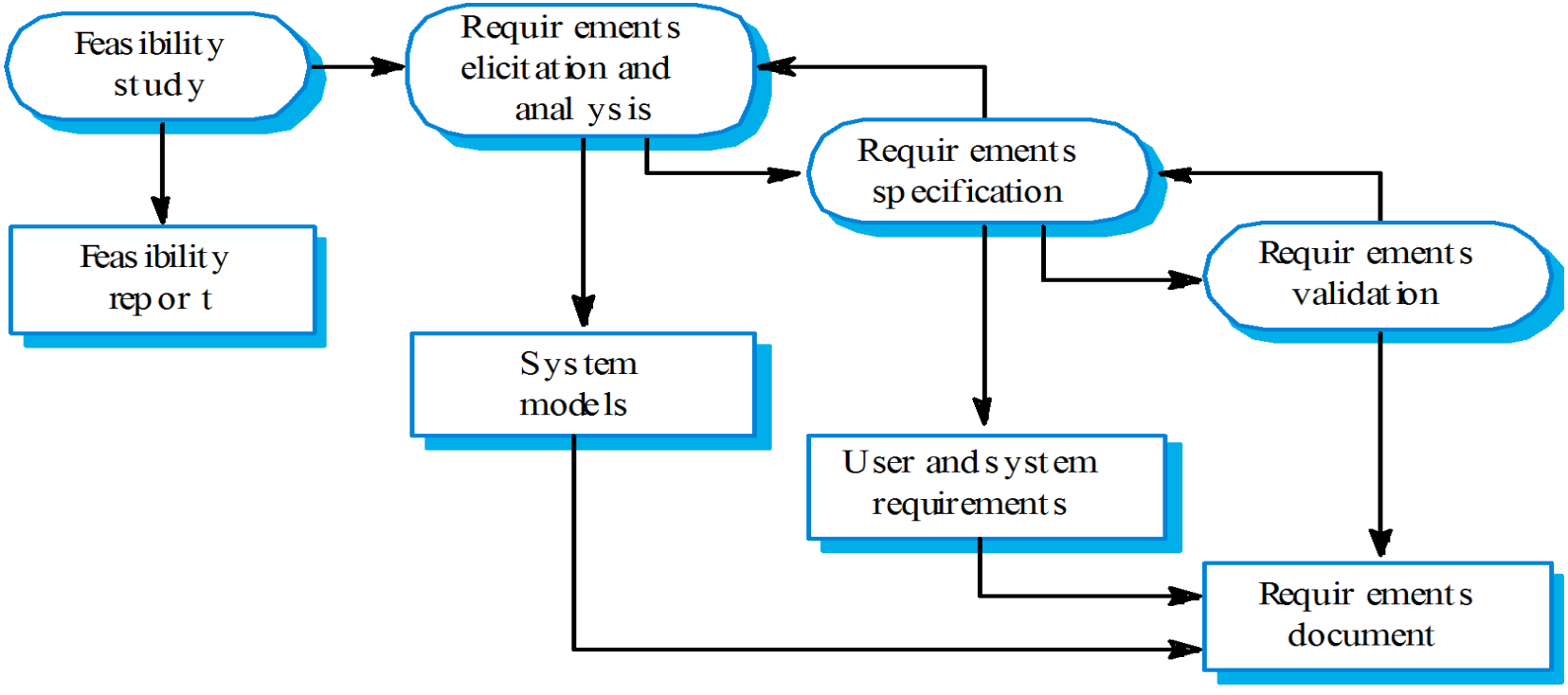
- Fit criteria form *objective standards* for judging whether a *proposed solution satisfies the requirements*:
  - It is easy to set fit criteria for *quantifiable requirements*.
  - It is hard for *subjective quality requirements*.

- Three ways to help *make requirements testable*:
  - Specify a quantitative description for *each adverb* and *adjective*.
  - Replace pronouns with *specific names of entities*.
  - Make sure that *every noun* is defined in exactly one place in the *requirements documents*.

The Requirement Engineering Process

- The process of establishing what services are required and the constraints on the system's operation and development
- Requirements engineering help software engineers to better understand the problem they will work to solve. It encompasses the set of tasks that lead to an understanding of what the business impact of the software will be, what the customer wants and how end-users will interact with the software.
  - **Requirement Engineering Process**
    - Feasibility Study
    - Requirements elicitation and analysis
    - Requirements Specification
    - Requirements Validation

The Requirements Engineering Process



Requirements Elicitation

- It is the practice of obtaining the requirements of a system from users, customers and other stakeholders. The practice is also sometimes referred to as **Requirement gathering**.
- Requirements elicitation practice include the following:
  - Interviews
  - Questionnaires
  - User observation
  - Workshops
  - Brain storming
  - Use cases
  - Role playing
  - And prototyping

Cont..

Requirements elicitation includes the following activities:

1. **Identifying actors**. During this activity, developers identify the different types of users the future system will support.
2. **Identifying scenarios**. During this activity, developers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
   – scenarios are concrete examples illustrating a **single case,**
3. **Identifying use cases:** developers derive from the scenarios a set of use cases that completely represent the future system.
   » use cases are abstractions describing all **possible cases.**

Cont…

4.**Refining use cases:** developers ensure that the system specification is **complete**, by **detailing** each use case and describing the **behavior of the system** in the presence of errors and exceptional conditions.

5.**Identifying relationships among use cases:** developers consolidate the use case model by eliminating redundancies. This ensures that the system specification is consistent.

6.**Identifying nonfunctional requirements:** developers, users, and clients agree on aspects that are visible to the user but not directly related to functionality.

These include constraints on the performance of the system, its documentation, the resources it consumes, its security, and its quality.

Scenarios

- Scenarios are real-life examples of how a system can be used.
- They should include
  - A description of the starting situation;
  - A description of the normal flow of events;
- What can go wrong and how this is handled
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

Example 1:LIBSYS scenario (1)

Initial assumption: The user has logged on to the LIBSYS system and has located the journal containing the copy of the article.

Normal: The user selects the article to be copied. He or she is then prompted by the system to either provide subscriber information for the journal or to indicate how they will pay for the article. Alternative payment methods are by credit card or by quoting an organisational account number.

The user is then asked to fill in a copyright form that maintains details of the transaction and they then submit this to the LIBSYS system.

The copyright form is checked and, if OK, the PDF version of the artic le is downloaded to the LIBSYS working area on the user computer and the user is informed that it is available. The user is asked to select a printer and a copy of the article is printed.

LIBSYS scenario (2)

What can go wrong: The user may fail to fill in the copyright form correctly. In this case, the form should be re-presented to the user for correction. If the resubmitted form is still incorrect then the user request for the article is rejected. The payment may be rejected by the system. The user request for the article is rejected. The article download may fail. Retry until successful or the user terminates the session.

It may not be possible to print the article. If the article is not flagged as print-only then it is held in the LIBSYS workspace. Otherwise, the article is deleted and the user's account credited with the cost of the article .

Other activities: Simultaneous downloads of other articles.

System state on completion: User is logged on. The downloaded article has been deleted from LIBSYS workspace if it has been flagged as print-only.

Use cases

- A scenario-based technique in UML which identify the actors in an interaction and which describe the interaction itself.
- Used also to clarify the system boundaries.
- A set of use cases should describe all possible interactions with the system.

Conti…

- Use case diagram:
  - An *actor* initiates a *use case.*
  - A *use case* defines a sequence of *interactions* between the actor and the system.
  - The *system* is depicted as a box.
  - Communication associations connect *actors* with the *use cases* in which they participate.
  - Relationships among use cases are defined by means of *include* and *extend* relationships.

Scenario for collecting medical history in MHC-PMS

What can go wrong: The patient's record does not exist or cannot be found. The nurse should create a new record and record personal information.

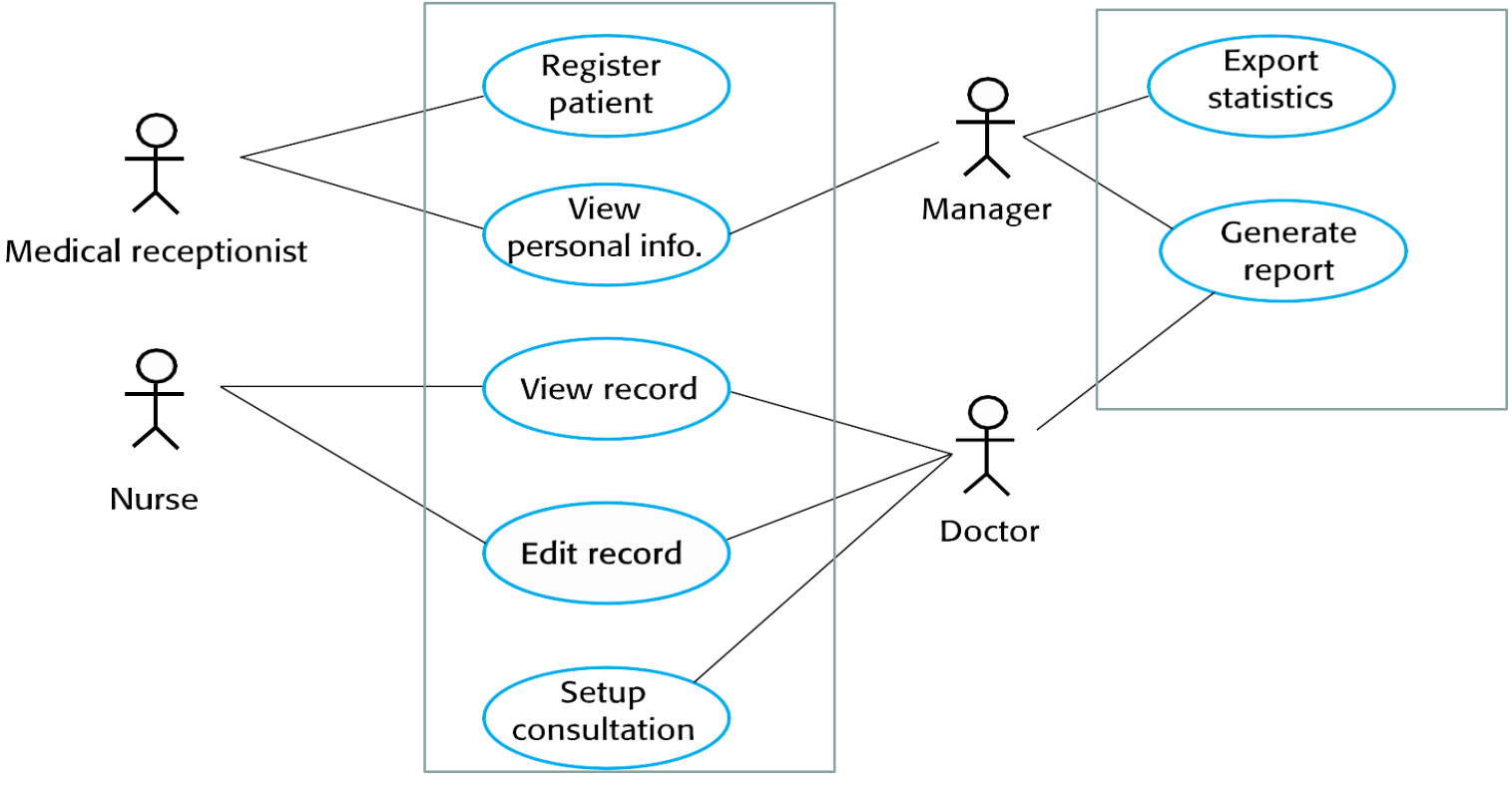
Patient conditions or medication are not entered in the menu. The nurse should choose the 'other' option and enter free text describing the condition/medication.

Patient cannot/will not provide information on medical history. The nurse should enter free text recording the patient's inability/unwillingness to provide information. The system should print the standard exclusion form stating that the lack of information may mean that treatment will be limited or delayed. This should be signed and handed to the patient.

Other activities: Record may be consulted but not edited by other staff while information is being entered.

System state on completion: User is logged on. The patient record including medical history is entered in the database, a record is added to the system log showing the start and end time of the session and the nurse involved.

Use cases for the MHC-PMS



Requirements Elicitation

- **Problems of Requirement Elicitation**
  - **Problems of scope:** The boundary of system is ill- defined. Or unnecessary details are provided.
  - **Problems of understanding:** The users are not sure of what they need, and don't have full understanding of the problem domain.
  - **Problems of volatility:** the requirements change overtime.

Requirements Elicitation

- **Guidelines of Requirements Elicitation**
  - Assess the business and technical feasibility for the proposed system
  - Identify the people who will help specify requirements.
  - Define the technical environment (e.g. computing architecture, operating system, telecommunication needs) into which the system or product will be placed
  - Identify "domain constraints" (i.e. characteristics of the business environment specific to the application domain) that limit the functionality or performance of the system or product to build
  - Define one or more requirements elicitation methods (e.g. interviews, team meetings, ..etc)
  - Solicit participation from many people so that requirements are defined from different point of views.
  - Create usage scenarios of use cases to help customers/ users better identify key requirements.

Problems of requirements elicitation

- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment change.

Requirements Analysis

- Requirements Analysis, determining whether the stated requirements are clear, complete, consistent and unambiguous.

Requirements Analysis

- **Stakeholder Identification**
  - Stakeholders are people or organizations that have a valid interest in the system. They may be affected by it directly or indirectly.
  - Stake holders may include:
    - Anyone who operates the system
    - Anyone who benefits from the system
    - Anyone involved in purchasing or procuring the system
    - People opposed to the system (negative

Requirements Analysis

- **Stakeholder Interviews**
  - Interviews are a common technique used in requirement analysis.
  - This technique can serve as a means of obtaining the highly focused knowledge from different stakeholders perspectives

Requirements Analysis

- **Types of Requirements:**
  - Customer Requirements:
    - Operational distribution or deployment: Where will the system be used?
    - Performance and related parameters: What are the critical system parameters to accomplish the mission?
    - Utilization environments: how are the various system components to be used?
    - Effectiveness requirements: How effective or efficient must the system be in performing its mission?
    - Operational life cycle: How long will the system be in use by the user?
    - Environment: what environments will the system be expected to operate in an effective manner?

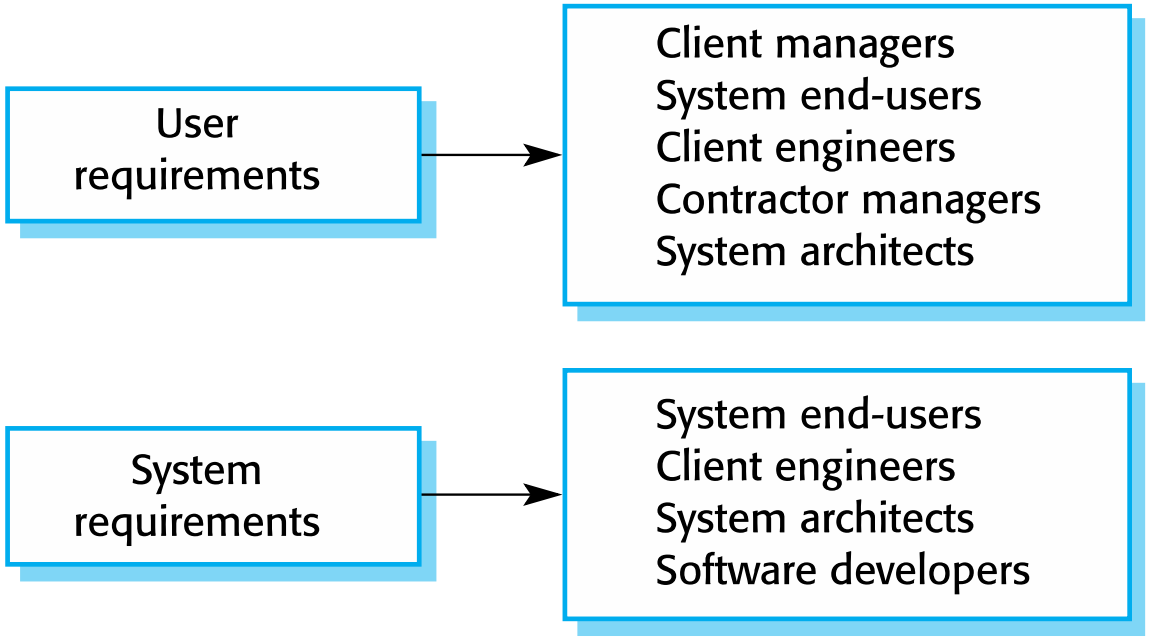
Requirements Analysis

- **Types of Requirements:**
  - Architectural Requirements:
    - A formal description and representation of a system, organized in a way that support reasoning about the structure of the system which comprises system components, the externally visible properties of those components, the relationships and the behavior between them, and provides a plan from which products can be procured and systems developed, that will work together to implement

Requirements Analysis

- **User requirements**
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints.
    - Written for customers.
- **System requirements**
  - A structured document setting out detailed descriptions of the system's:
    - functions,
    - Services,
    - operational constraints.
  - **Defines what should be implemented**
    - → So, it may/will be part of a contract between client and contractor.

Readers of different types of requirements specification



Requirements Analysis

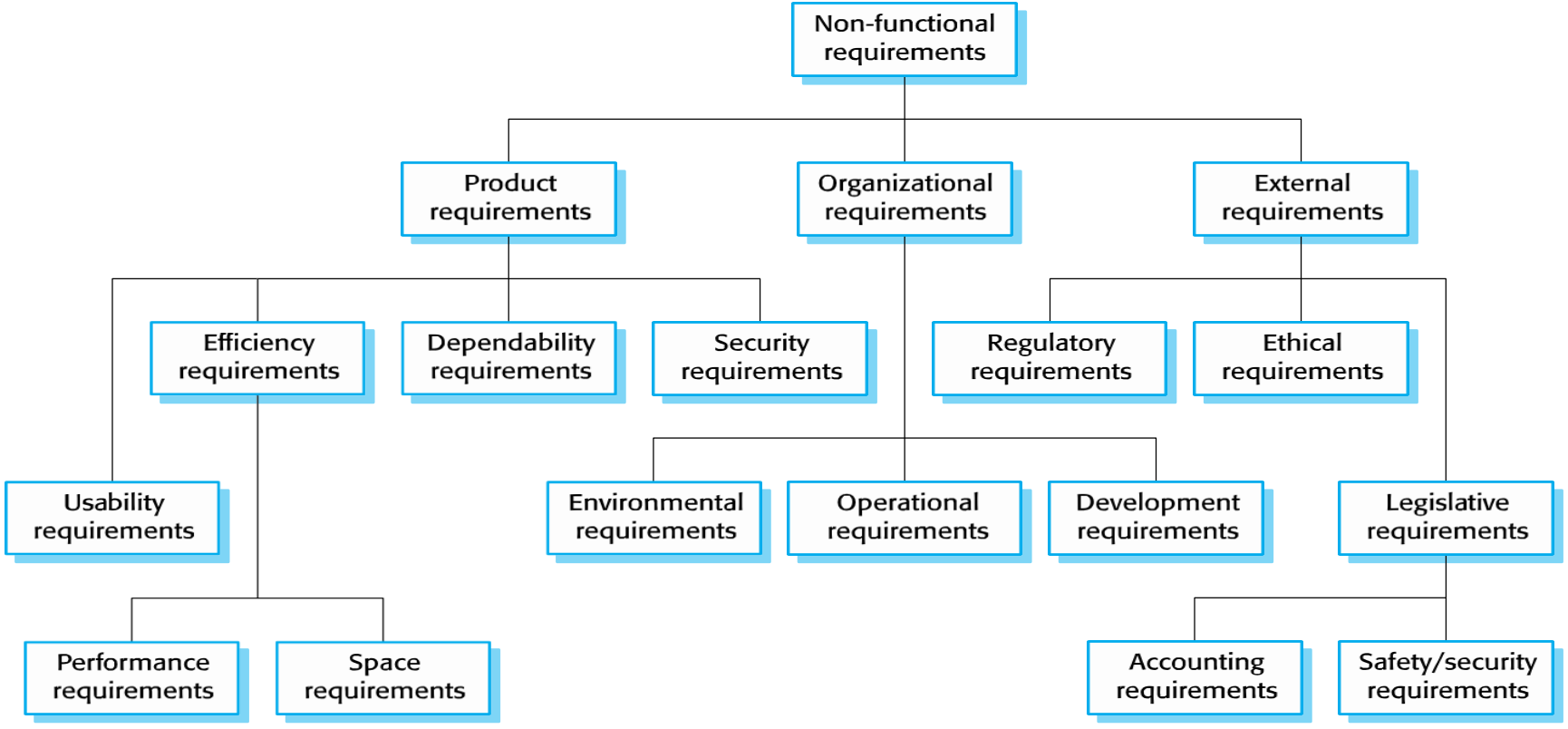
- **Types of Requirements:**
  - **Functional Requirements:**
    - Defines functions of a software system or its components. They may be calculations, technical details, data manipulation and processing and other specific functionality that define "what a system is supposed to accomplish?"
    - They describe particular results of a system
    - Functional requirement are supported by non-functional requirement

Requirements Analysis

– Non-Functional Requirements:

- They are requirements that specify criteria that can be used to judge the operation of a system, rather than specific behavior.
- Functional requirements define what the system is supposed to ***do***, whereas non-functional requirements define how a system is supposed to ***be***.
- Non-functional requirements can be divided into two main categories:
  - Execution qualities, such as security and usability, which are observable at runtime.
  - Evolution qualities, such as testability, maintainability and scalability.

Types of nonfunctional requirement



Metrics for specifying nonfunctional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Cont…

Example:

- **Idea:** ***A Library Management System should be designed. Information on books, CDs, DVDs, Journals, etc. can be stored and retrieved.***
- **Possible Requirement**
  - Searching by Title, Author, and/or ISDN should be possible
  - User interface should be web based (accessing via www browser)
  - At least 20 Transaction per second performed
  - All services should be available within 10 minutes
  - Users have no access to personal data of other users

Problem statement

Functional Req.

Implementation Req.

Performance Req.

Availability Req.

Security req.

Requirements Specifications

- Requirements Specification is the direct result of a requirement analysis and can refer to:
  - Software Requirements Specification
  - Hardware Requirements Specification

Requirements Specifications

- A Software Requirements Specification (SRS) –is a complete description of the behavior of a system to be developed.
- It includes a set of use cases that describe all the interactions the users will have with the software. In addition to use cases, the SRS also contains non- functional requirements (such as performance requirements, quality standards, or design constraints)

Conti…

- A Software Requirements Specification (SRS)
  - The software requirement specification document enlists all necessary requirements for project development. To derive the requirements we need to have clear and thorough understanding of the products to be developed.
  - A general organization of an SRS is as follows:
    - Introduction
      - Purpose, Scope, Definitions, System Overview, References
    - Overall Description
      - Product Perspective, Product functions, User characteristics, constraints, assumptions and dependencies.
    - Specific Requirements
      - External Interface requirements, functional requirements, performance requirements, design constraints, logical database requirement, software system attributes.

Requirements Validation and Verification

- Validation & Verification), is the process of checking whether the requirements, as identified, do not contradict the expectations about the system of various stakeholders and do not contradict each other.
- It is Requirements Quality Control

Validation Vs. Verification

- **Validation**: "Am I building the right product?" checking a work product against higher-level work products or authorities that frame this particular product.
  - Requirements are validated by stakeholders
- **Verification**: "Am I building the product right?" checking a work product against some standards and conditions imposed on this type of product and the process of its development.
  - Requirements are verified by the analysts mainly

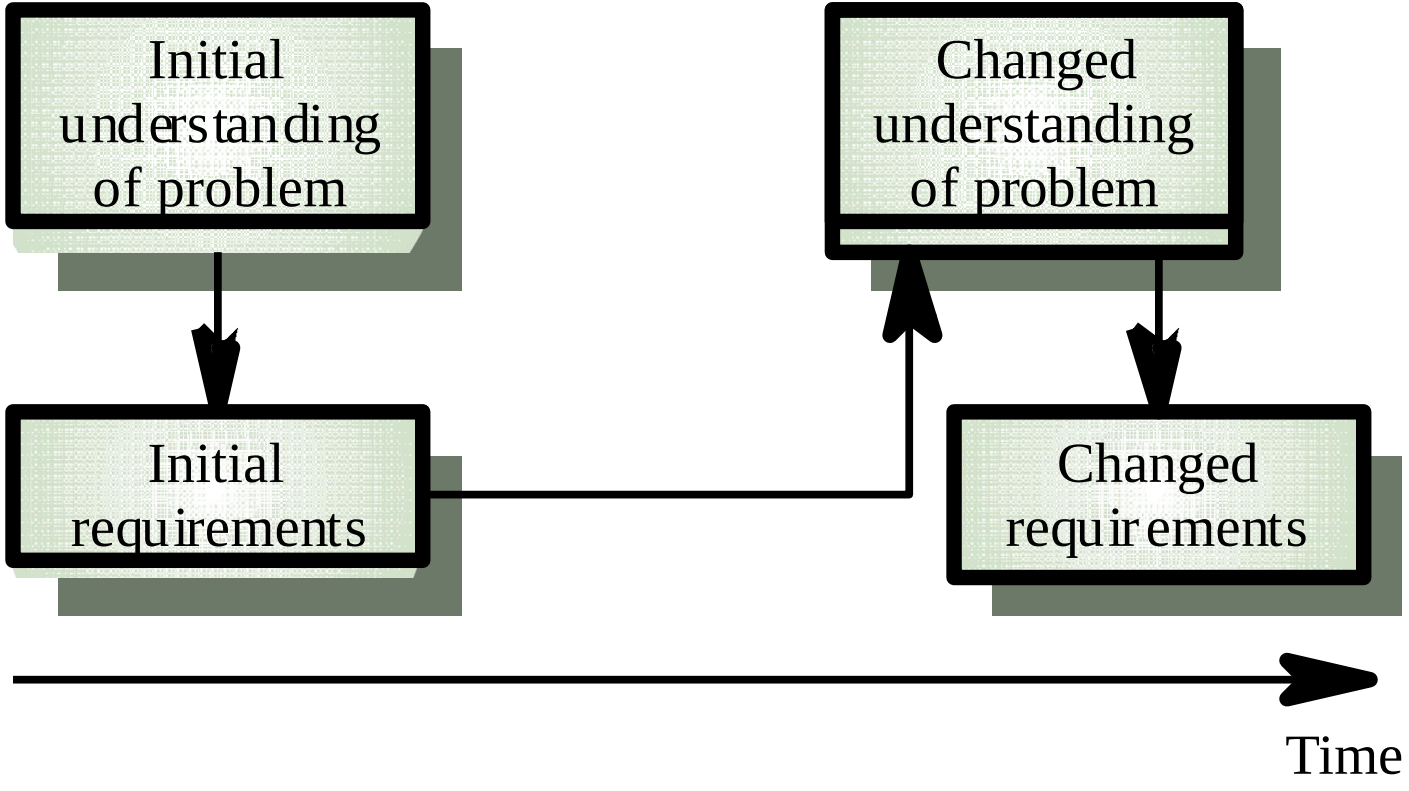
Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development
- Requirements are inevitably incomplete and inconsistent
  - New requirements emerge during the process as business needs change and a better understanding of the system is developed
  - Different viewpoints have different requirements and these are often contradictory

Requirements change

- The priority of requirements from different viewpoints changes during the development process
- System customers may specify requirements from a business perspective that conflict with end-user requirements
- The business and technical environment of the system changes during its development

Requirements evolution



Requirement Management

–Set of activities that help project team to identify, control, and track requirements and changes as project proceeds

–Requirements begin with identification. Each requirement is assigned a unique identifier. Once requirement have been identified, traceability table are developed.

Traceability Table:

– **Features traceability table** - shows how requirements relate to customer observable features

–**Source traceability table** - identifies source of each requirement

–**Dependency traceability table** - indicate relations among requirements

–**Subsystem traceability table** - requirements categorized by subsystem

–**Interface traceability table** - shows requirement relations to internal and external interfaces

It will help to track, if change in one requirement will affect different aspects of the system.



System models

- Different models may be produced during the requirements analysis activity
- Requirements analysis may involve three structuring activities which result in these different models
  - Partitioning. Identifies the structural (part-of) relationships between entities
  - Abstraction. Identifies generalities among entities
  - Projection. Identifies different ways of looking at a problem

What is UML?

- It’s an industry standard graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software intensive system.” [Booch]
- UML is a generic broad language enabling the key aspects of software development to capture on paper.
- General-purpose modeling language
- It’s a notation not a methodology.

What is UML?

- System development focuses on three different models of the system:
  - The **functional model:** represented in UML with use case diagrams, describes the **functionality of the system** from the user's point of view.
  - **The object model:** represented in UML with class diagrams, describes the **structure of a system** in terms of objects, attributes, associations, and operations.
  - The **dynamic model:** represented in UML with sequence diagrams, state chart diagrams, and activity diagrams, describes the **internal behavior of the system.**

Benefits of UML

- Use graphical notation to communicate more clearly than natural language (imprecise) and code(too detailed).
- Quick understanding
- Help acquire an overall view of a system.
- Lessens chances of conflicts.
- UML is not dependent on any one language or technology.
- Necessary to manage complexity.
- UML moves us from fragmentation to standardization.

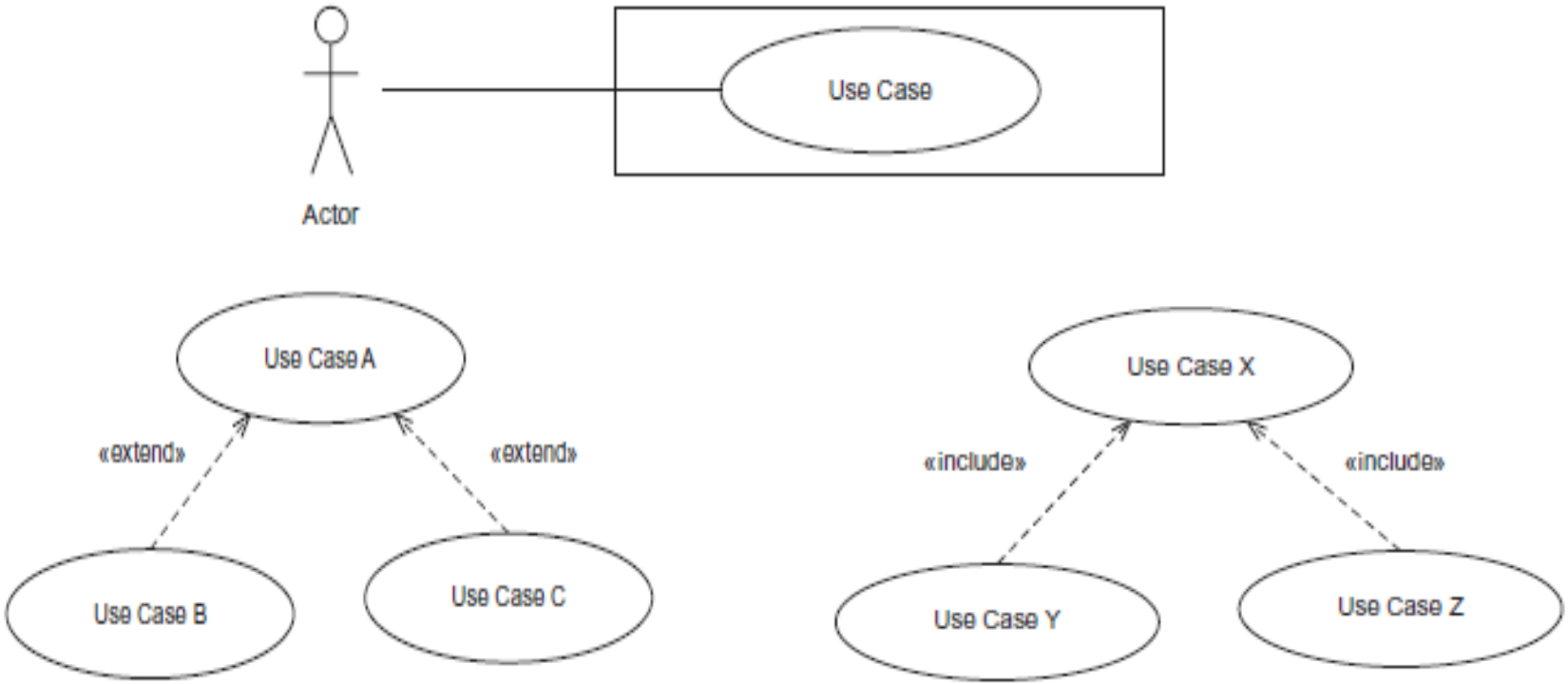
Use case diagram

- A **use case** is the description of set of interaction between an actor and system.
- Derived from use-case study scenarios. It is an overview of use cases, actors, and their communication relationships to demonstrate how the system reacts to requests from external users. It is used to capture system requirements.

Components of use case diagram

1. **Actor:-** any entity that interacts with the system (e.g., a user, another system, the system's physical environment).
2. **Use case:-** A set of scenarios that describe an interaction between a user and a system, including alternatives
3. **System boundary:-** rectangle diagram representing the boundary between the actors and the system.

Use case diagram



UML notation for a use case diagram

Identify actor, use case and system boundary

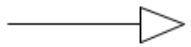
Example: university registration system

1. Professors indicate which courses they will teach on-line.
2. A course catalog can be printed
3. Allow students to select on-line four courses for upcoming semester.
4. No course may have more than 10 students or less than 3 students.
5. When the registration is completed, the system sends information to the billing system.
6. Professors can obtain course rosters on-line.
7. Students can add or drop classes on-line.

Types of relationships in use case diagram

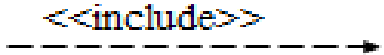
1. **Association:-** communication between an actor and a use case.
   - Represented by a solid line.

2. **Generalization:-** relationship between one general use case and a special use case.
   - Represented by a line with a triangular arrow head toward the parent use case.



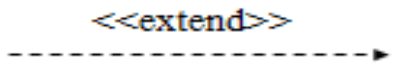
Types of relationships in use case diagram

3. **Include:-** occurs when a chunk of behavior is similar across more than one use case.
   - Use “include” in stead of copying the description of that behavior.
   - The base use case is incomplete without the included use case.
   - The included use case is mandatory and not optional.

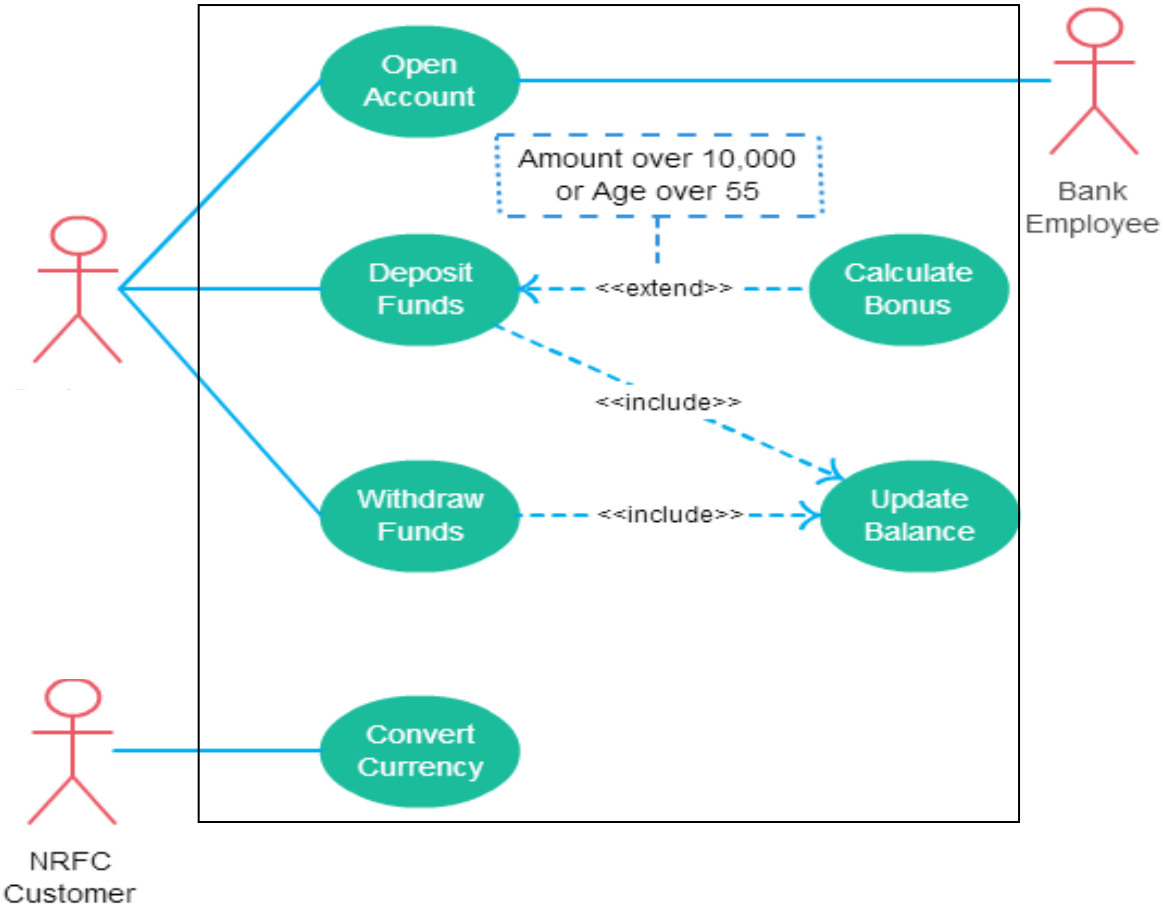


   - The direction of a <<includes>> relationship is to the using use case

4. **Extend:-** The extending use case may add behavior to the base use case.
   - The direction of an <<extends>> relationship is to the extended use case



Conti…


Open Account
Amount over 10,000 or Age over 55
Bank Employee
Deposit Funds
<<extend>>
Calculate Bonus
<<include>>
Withdraw Funds
<<include>>
Update Balance
Convert Currency
NRFC Customer

Use case Description

- Each Use Case contains a full set of textual details about the interactions and scenarios contained within it.
  - **Use Case:** Use Case Name
  - **Short Description:** A Brief Description of the Use Case
  - **Participating Actors**
  - **Pre-Conditions:** A description of the conditions that must be satisfied before the use case is invoked
  - **Post-Conditions :** A description of what has happened at the end of the use case
  - **Main Flow:** A list of the system interactions that take place under the most common scenario. For example, for withdraw money , this would be enter card, enter pin, etc...
  - **Alternate Flow(s):** A description of possible alternative interactions.
  - **Exception Flow(s):** A description of possible scenarios where unexpected or unpredicted events have taken place

Use case Description

Example: read the following scenario and identify actors and use cases plus to the use case description.

- Person who owns one or more Accounts in the Bank wants to withdraw money.
- The Bank Customer specifies an Account and provides credentials to the Bank proving that s/he is authorized to access the Bank Account.
- The Bank Customer specifies the amount of money s/he wishes to withdraw.
- The Bank checks if the amount is consistent with the rules of the Bank and the state of the Bank Customer's account. If that is the case, the Bank Customer receives the money in cash.

Use case Description

Use case ID	UC-01
Use case name	WithdrawMoney
Actor	Bank customer
Use case description	
Pre-condition	• Bank Customer has opened a Bank Account with the Bank and • Bank Customer has received an ATM Card and PIN
Post-condition	• Bank Customer has the requested cash or • Bank Customer receives an explanation from the ATM about why the cash could not be dispensed
Main flow	1. The Bank Customer inputs the card into the ATM. 2. The ATM requests the input of a four-digit PIN. 3. The Bank Customer types in PIN 4. If several accounts are recorded on the card, the ATM offers a choice of the account numbers for selection by the Bank Customer. 5. The Bank Customer selects an account. 6. If only one account is recorded on the card or after the selection, the ATM requests the amount to be withdrawn. 7. The Bank Customer inputs an amount. 8. The ATM outputs the money and a receipt and stops the interaction
Alternative flow	
Exception flow	1. The Bank Customer inputs her card into the ATM.[Invalid card] 2. The Bank Customer types in PIN. [Invalid PIN] 3. The Bank Customer selects an account. 4. The Bank Customer inputs an amount. [Amount over limit]

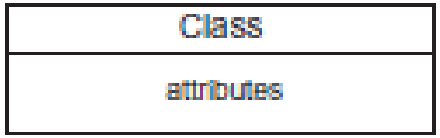
Class Diagram

- Class diagrams can be derived from use-case diagrams or from text analysis of the given problem domain.
- Used to define the static structure of classes in a system and their associations
- Generated by system analysts and designers and will be iteratively refined in the subsequent phases during the software development life cycle.
- The class box always holds the class name.
- Optionally, the *attributes* and *operations* of a class may also be depicted.
- To distinguish between a class (the type) and an object (an instance of the type), an object name is shown *underlined*

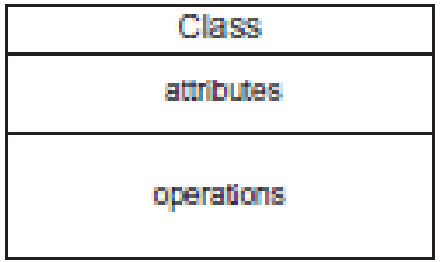
Class Diagram



Class



Class with attributes



Class with attributes and operations





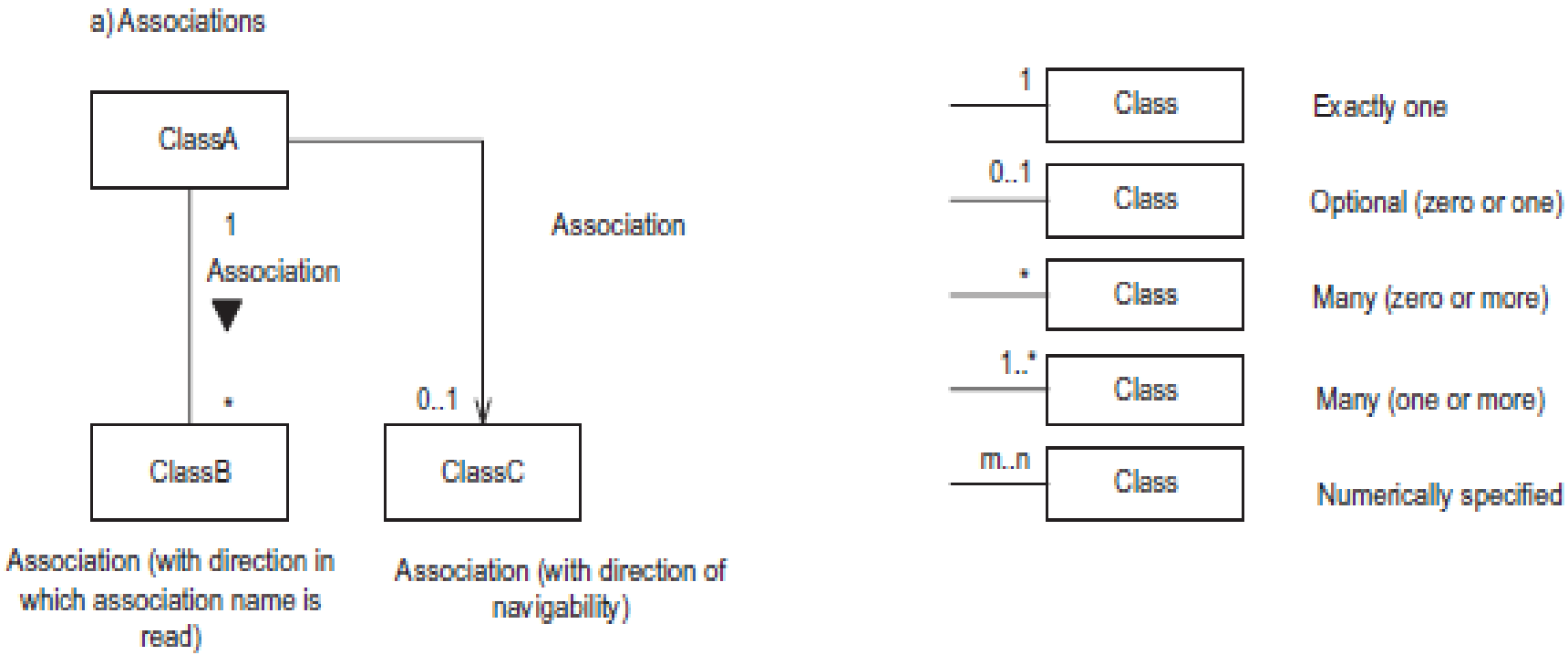


Objects

Class Diagram

- Associations:
  - Structural *relationship* between two or more classes depict as a line joining the two class.
  - Relationships b/n classes can be:
    - Association,
    - Whole/part relationship, and
    - Generalization and specialization .
  - On each end of the association line joining the classes is the *multiplicity* of the association.
  - The multiplicity of an association specifies *how many instances* of one class may relate to a single instance.

Class Diagram

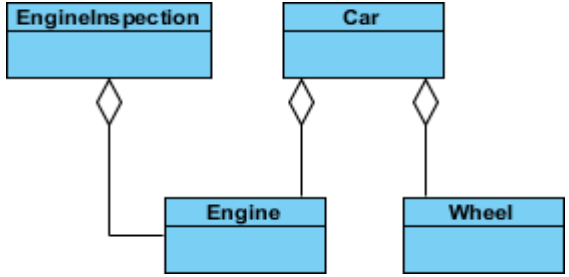


Conti…

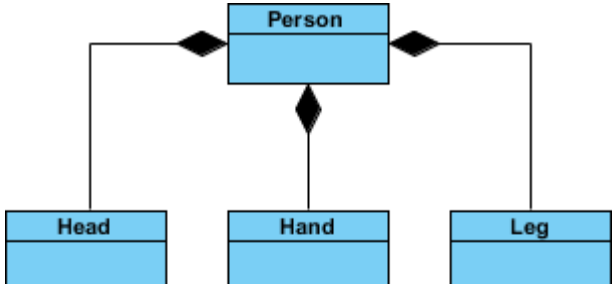
- Generalization/Specialization Hierarchies:
  - A generalization/specialization hierarchy is an *inheritance* relationship.
  - Depicted as an arrow joining the *subclass* (child) to the *superclass* (parent),

Conti…

- **Aggregation:** In an aggregation relationship, the associated objects exist independently within the scope of the system. Example A car needs a wheel for its proper functioning, but it may not require the same wheel. It may function with another wheel as well.

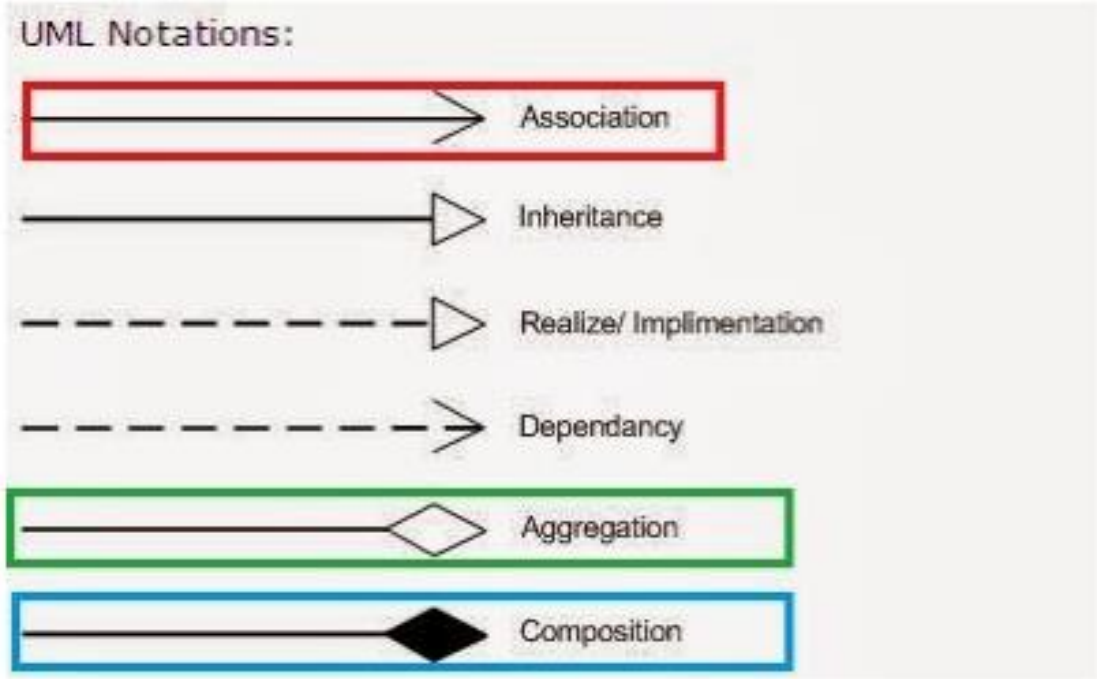


- **Composition:** is a strong form of aggregation in which the "whole" is completely responsible for its parts and each "part" object is only associated to the one whole object.

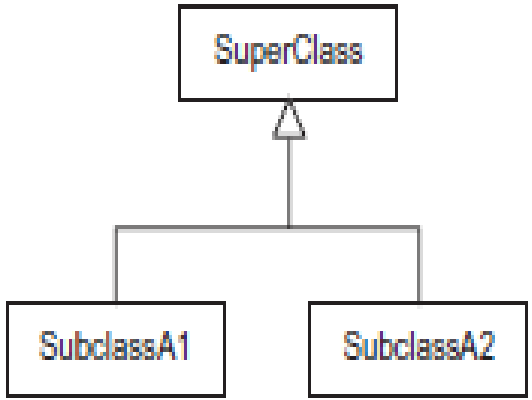


Conti…

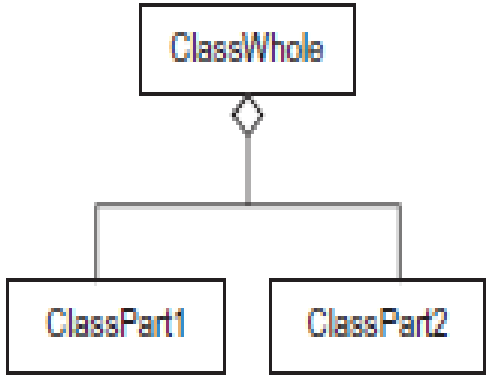
Here are UML notations for a different kind of dependency between the two classes.



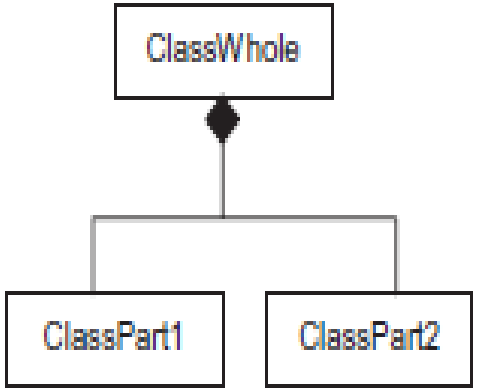
Example



Generalization/specialization



Aggregation hierarchy



Composition hierarchy

1. Composition:

Since Engine is-part-of Car, the relationship between them is Composition. Here is how they are implemented between Java classes.

```
public class Car {

    //final will make sure engine is initialized
    private final Engine engine;

    public Car(){
       engine  = new Engine();
    }
}

class Engine {
    private String type;
}
```

2. Aggregation:

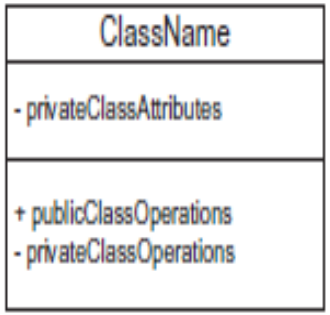
Since `Organization` has `Person` as employees, the relationship between them is Aggregation. Here is how they look like in terms of Java classes

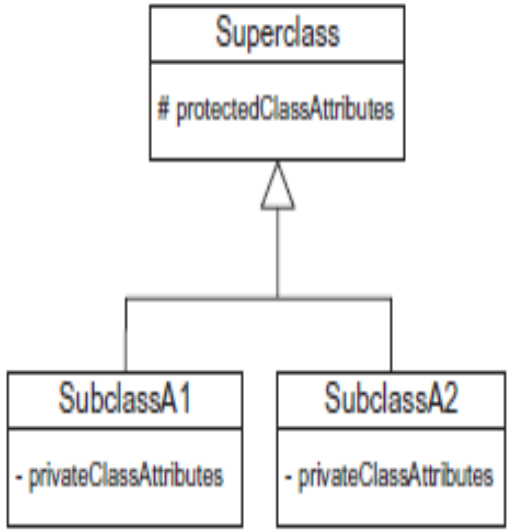
```
public class Organization {
    private List employees;
}

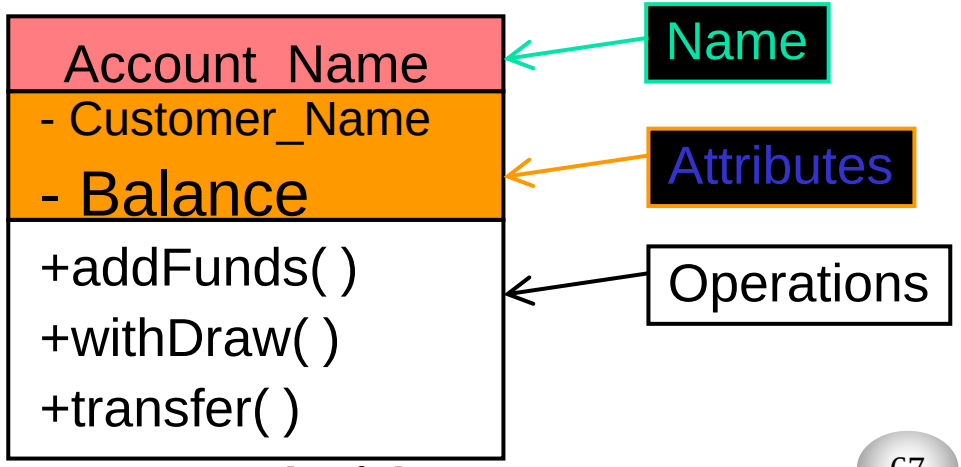

public class Person {
    private String name;
}
```

Class diagram Visibility

- Visibility refers to whether an *element* of the class is visible from outside the class.
- Depicting visibility is *optional* on a class diagram.
- *Public* visibility, denoted with a + symbol (visible outside)
- *Private* visibility, denoted with a – symbol (visible only in the class)
- *Protected* visibility, denoted with a # symbol (class + subclass)





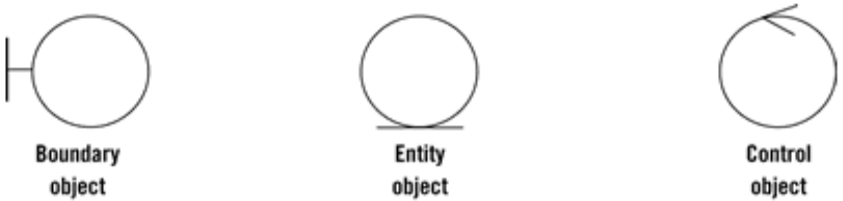


Example of class representation

Sequence Diagram

- It to show sequence of operations or how processes operate with one another and in what order. It is a construct of a Message Sequence Chart.
- It shows object interactions arranged in time sequence.
- It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development.

Conti…



Entities *(model)*
Objects representing system data, often from the domain model.

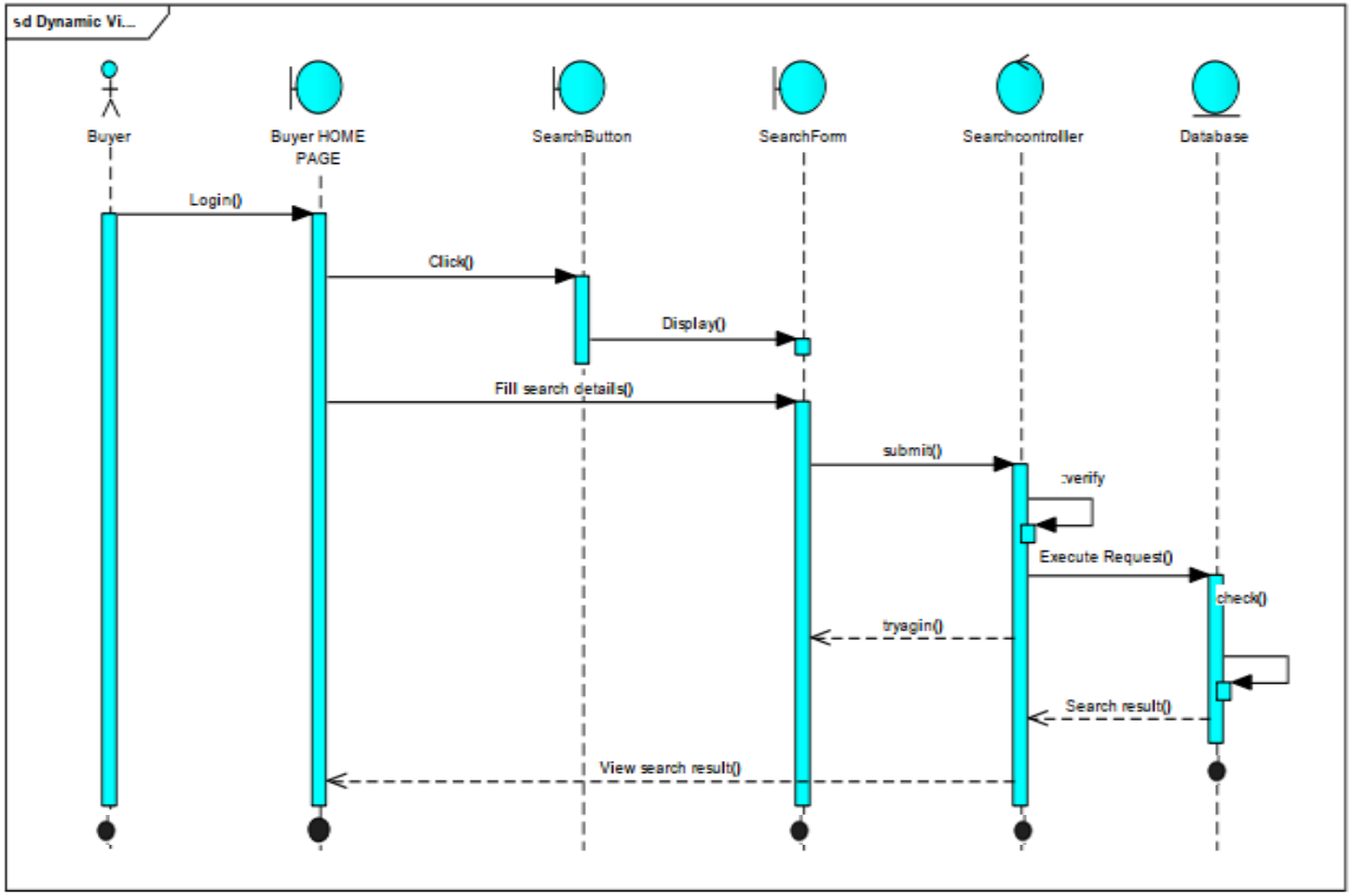
Boundaries *(view/service collaborator)*
Objects that interface with system actors (e.g. a **user** or **external service**). Windows, screens and menus are examples of boundaries that interface with users.

Controls *(controller)*
Objects that mediate between boundaries and entities. These serve as the glue between boundary elements and entity elements, implementing the logic required to manage the various elements and their interactions. It is important to understand that you may decide to implement controllers within your design as something other than objects – many controllers are simple enough to be implemented as a method of an entity or boundary class for example.

Four rules apply to their communication:

1. Actors can only talk to boundary objects.

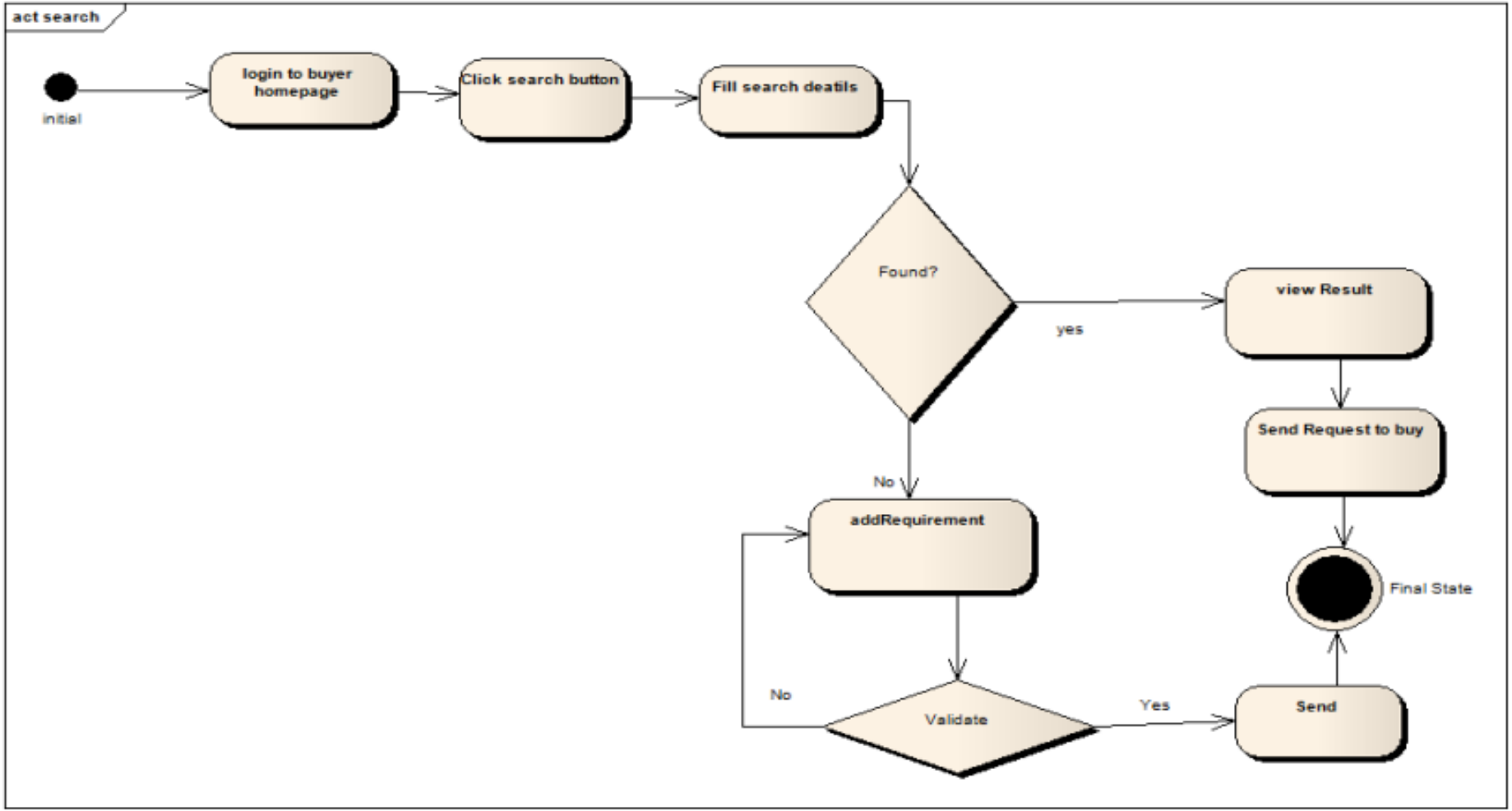
Example Sequence diagram for Search Use case



Activity Diagram

- Outline of activity - data and control flow among related objects. An activity is an action for a system operation or a business process, such as those outlined in the use-case diagram.
- It also covers decision points and threads of complex operation processes. It describes how activities are orchestrated to achieve the required functionality.

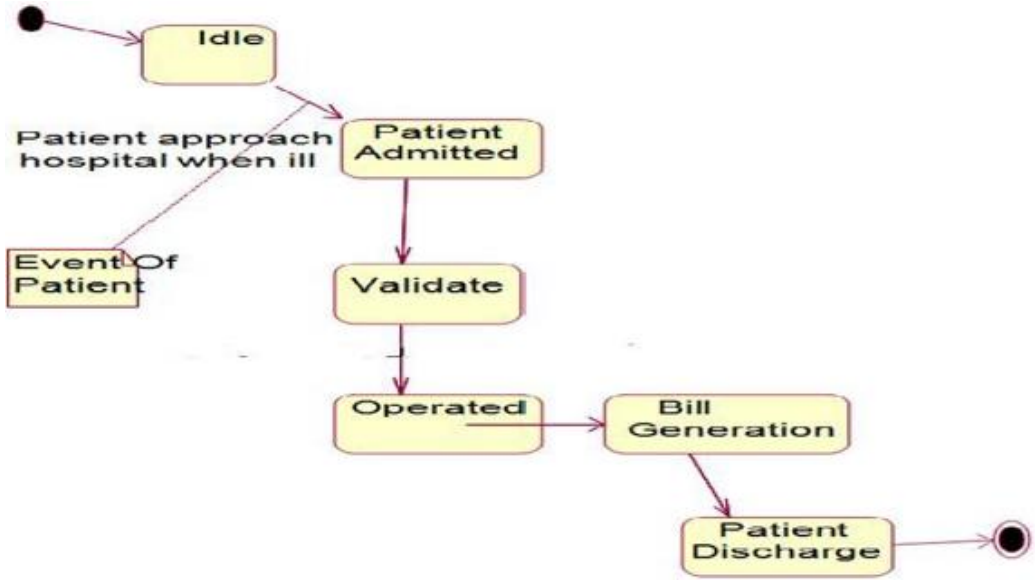
Activity Diagram



State Diagram

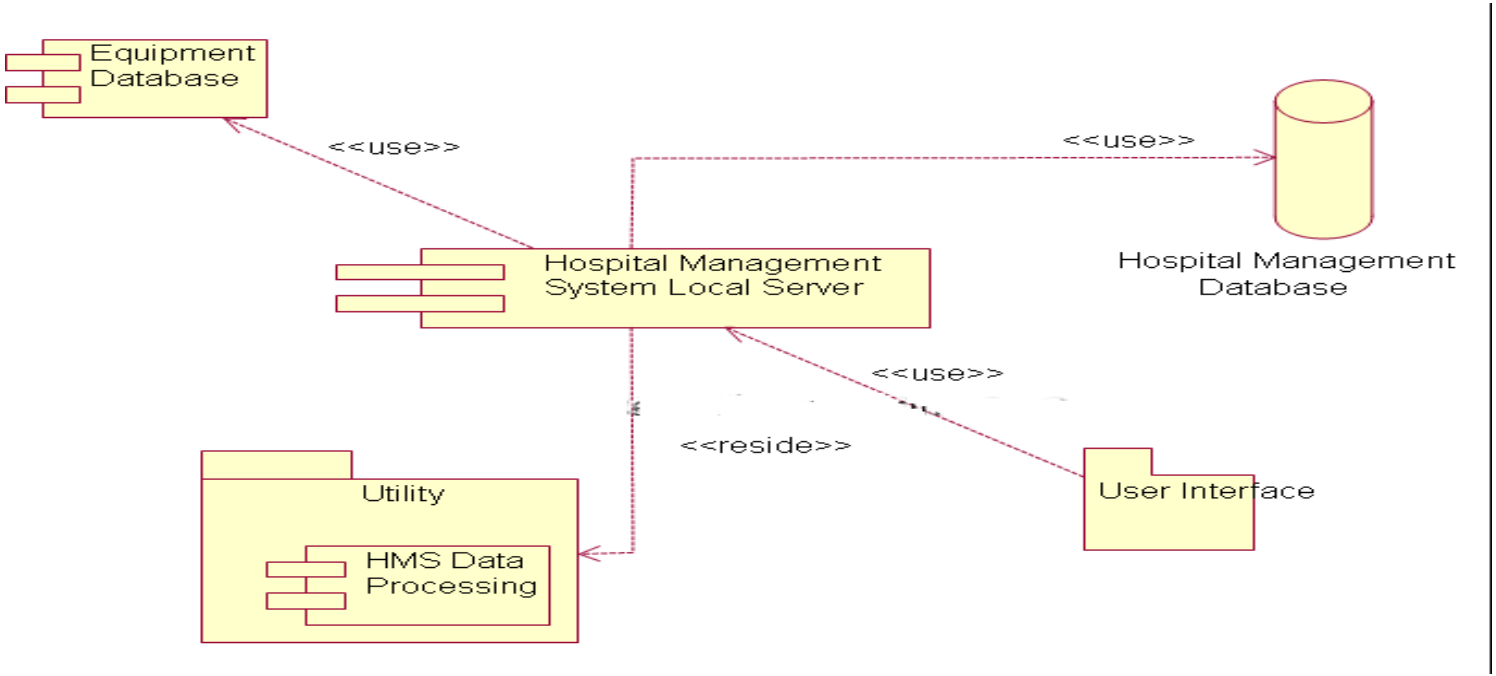
State chart diagram

- Describes the life cycle of objects using a finite state machine.
- The diagram consists of states and the transitions between states. Transitions are usually caused by external stimuli or events. They can also represent internal moves by the object.



Component diagram

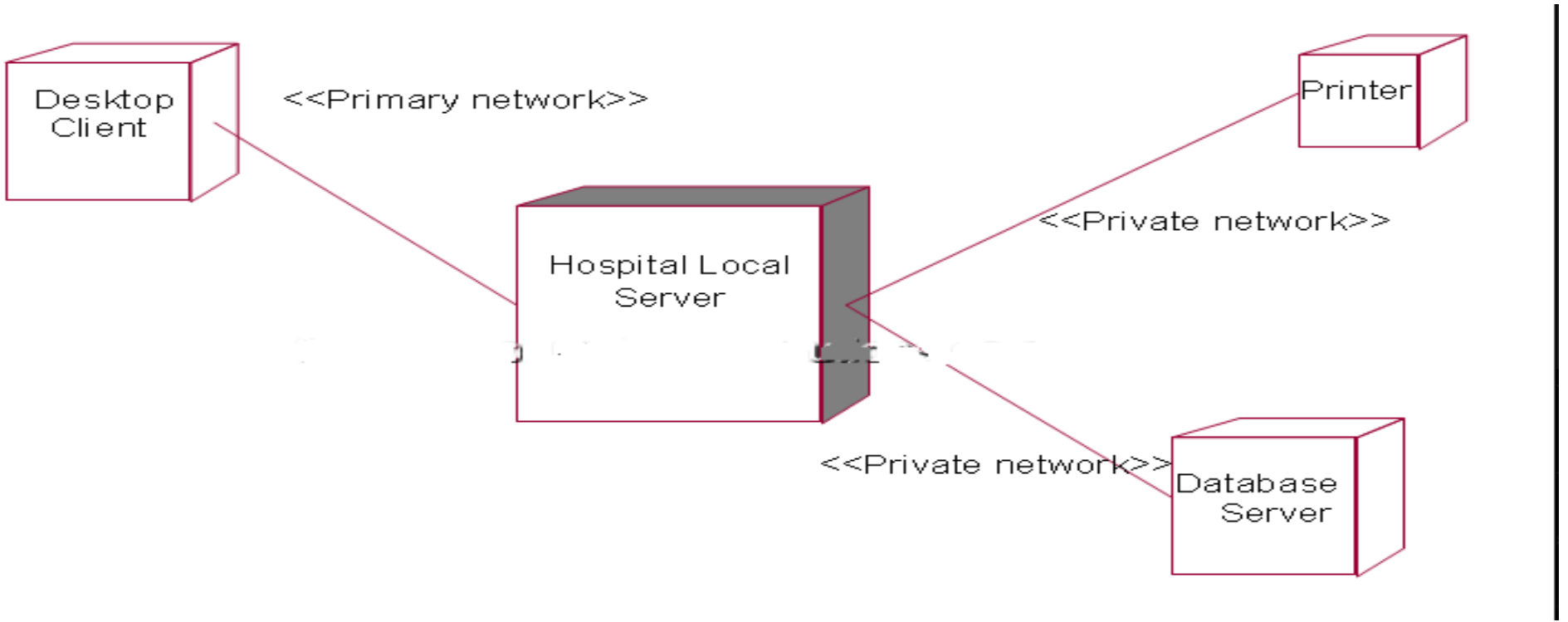
- It Describes all components in a system, their interrelationships, interactions, and the interface of the system.
- It is an outline of the composition structure of components or modules



Deployment Diagram

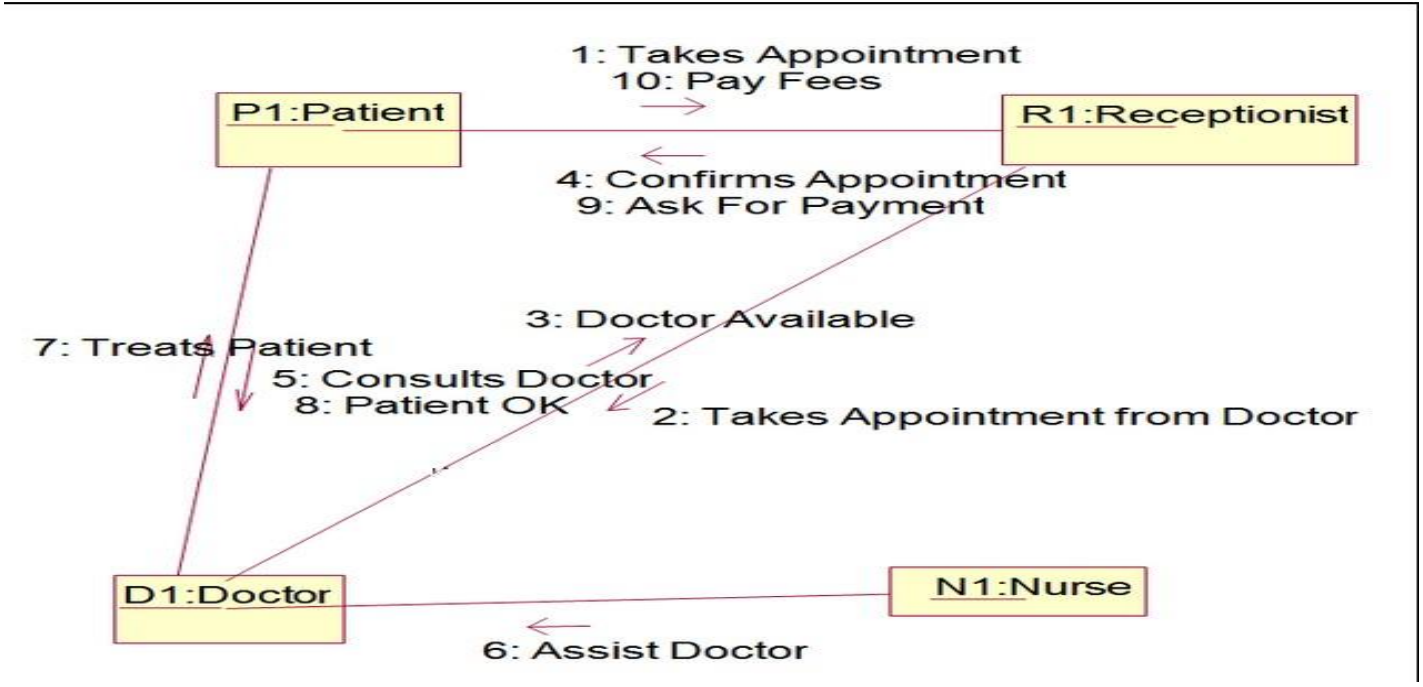
- Describes system hardware, software, and network connections for distributed computing.
- It covers server configuration and network connections between server nodes in real-world setting.

Deployment Diagram



Collaboration Diagram

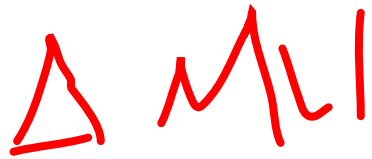
- Describes the sequence of message passing among objects in the system. Equivalent to sequence diagram, except that it focuses on the object—s role.
- Each communication link is associated with a sequence order number plus the passed messages.



SRS Document Template

1. Introduction to the Document
   - 1.1 Purpose of the Product
   - 1.2 Scope of the Product
   - 1.3 Acronyms, Abbreviations, Definitions
   - 1.4 References
2. General Description of Product
   - 2.1 Context of Product
   - 2.2 Product Functions
   - 2.3 User Characteristics
   - 2.4 Constraints
   - 2.5 Assumptions and Dependencies
3. Specific Requirements
   - 3.1 External Interface Requirements
     - 3.1.1 User Interfaces
     - 3.1.2 Hardware Interfaces
     - 3.1.3 Software Interfaces
     - 3.1.4 Communications Interfaces
   - 3.2 Functional Requirements
     - 3.2.1 Class 1
     - 3.2.2 Class 2
     - …
   - 3.3 Performance Requirements
   - 3.4 Design Constraints
   - 3.5 Quality Requirements
4. System Model
   - 4.1 Use Case Diagram
   - 4.2 Class Diagram

   …
1. Appendices



- IEEE Standard for *SRS Document*

END

Q?